

Flask Application Bug Fixes Documentation

Backend Project 2 (Debugging)

Introduction

During the development of a Flask-based notes application, several issues were encountered that prevented the app from functioning correctly. These bugs primarily involved routing, form handling, and data validation. This document outlines each problem, its cause, and the applied fix, along with the resulting behaviour after correction.

Bug 1: Home Route Only Allowed POST Method

Original Code: `@app.route('/', methods=["POST"])`

Problem: When accessing the application in a browser, Flask sends a GET request. Since only POST was allowed, the GET request was rejected, causing the application to crash.

Fix: `@app.route("/", methods=["GET", "POST"])`

Result: The page loads normally in the browser, and form submissions continue to work correctly.

Bug 2: Incorrect Way of Fetching Form Data

Original Code: `note = request.args.get("note")`

Problem: `request.args` is used for GET requests, but the form submits data via POST. This mismatch prevented notes from being captured.

Fix: `note = request.form.get("note")`

Result: Notes are correctly captured from the form submission.

Bug 3: HTML Form Missing Method Attribute

Original Code: `<form action="">`

Problem: The default method for forms is GET, but the backend expects POST. This caused a mismatch in data handling.

Fix: `<form method="POST">`

Result: Form data is sent correctly to the Flask backend.

Bug 4: Appending Empty Notes

Original: Behaviour Empty notes could be added to the list, resulting in unnecessary blank entries.

Fix:

if note:

```
    notes.append(note)
```

Result: Empty list items are prevented, ensuring only valid notes are stored.

Conclusion:

By addressing these five bugs, the Flask notes application now functions as intended. The routing supports both GET and POST requests, form data is correctly captured and data validation prevents empty entries. These fixes collectively improve the stability, usability, and reliability of the application.